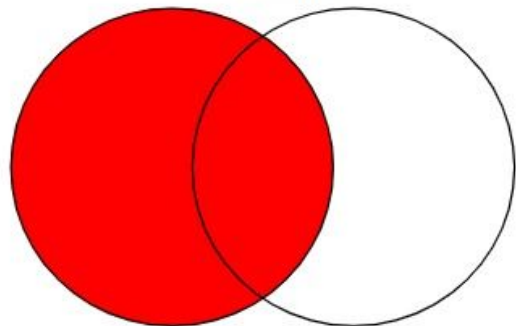


SQL (DML)

Clase 11

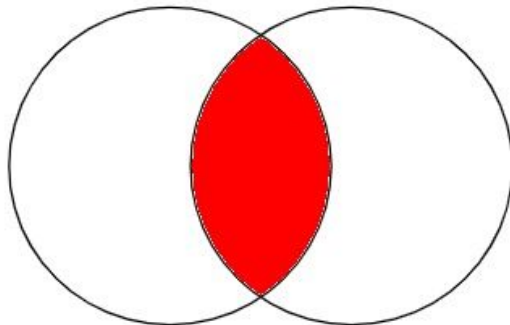
SQL JOINS: GUÍA RÁPIDA

LEFT JOIN



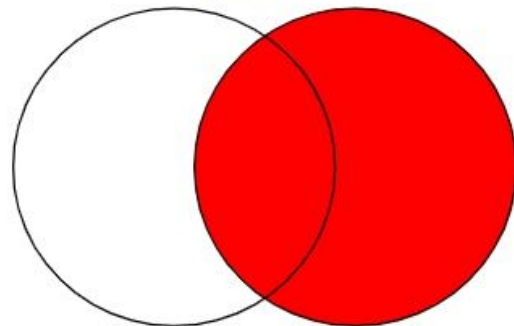
```
SELECT <columnas>  
FROM tablaA A  
LEFT JOIN TablaB B  
ON A.key = B.key
```

INNER JOIN



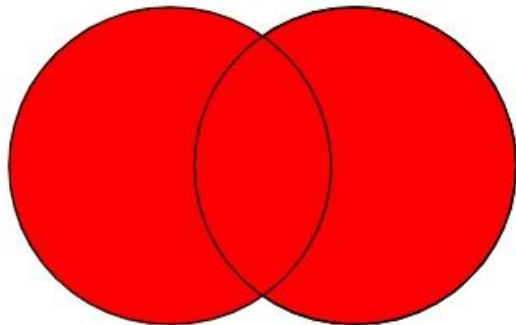
```
SELECT <columnas>  
FROM tablaA A  
INNER JOIN TablaB B  
ON A.key = B.key
```

RIGHT JOIN



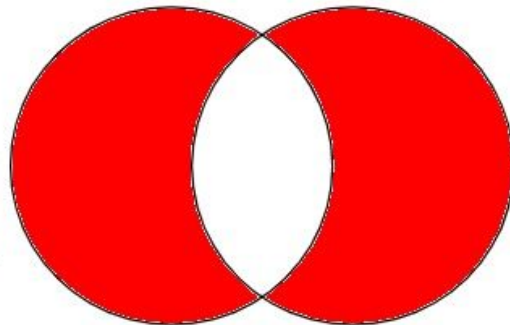
```
SELECT <columnas>  
FROM tablaA A  
RIGHT JOIN TablaB B  
ON A.key = B.key
```

FULL OUTER JOIN



```
SELECT <columnas>  
FROM tablaA A  
FULL OUTER JOIN TablaB B  
ON A.key = B.key
```

FULL OUTER JOIN



```
SELECT <columnas>  
FROM tablaA A  
FULL OUTER JOIN TablaB B  
ON A.key = B.key  
WHERE A.Key IS NULL OR  
B.Key IS NULL
```

LEFT JOIN, RIGHT JOIN

- LEFT JOIN: contiene todos los registros de la tabla de la izquierda, aún cuando no exista un registro correspondiente en la tabla de la derecha, para uno de la izquierda. Retorna un valor nulo (NULL) en caso de no correspondencia.
- RIGHT JOIN: es la inversa del LEFT JOIN.

tabla clientes

ID	Nombre
1	Juan
2	María
3	Pedro

tabla pedidos

ID	Producto
1	Camiseta
2	Pantalón

LEFT JOIN, RIGHT JOIN

SELECT clientes.ID, clientes.Nombre, pedidos.Producto

FROM clientes

LEFT JOIN pedidos

ON clientes.ID = pedidos.ID;

ID	Nombre	Producto
1	Juan	Camiseta
2	María	Pantalón
3	Pedro	NULL

UNION, EXCEPT, INTERSECT

- UNION: Combina los resultados de dos o más consultas y elimina cualquier duplicado en el proceso. No se mostrarán tuplas repetidas.
- EXCEPT: Se asemeja a la resta de conjuntos.
- INTERSECT: Se asemeja a la intersección de conjuntos.

Todos estos tipos de consultas deben tener la misma estructura de columna y el mismo tipo de datos.

Ejemplo

Mostrar el dni de los alumnos con dni mayor a 25000000 que no hayan rendido examen.

```
SELECT dni
```

```
FROM alumno
```

```
WHERE (dni > 25000000 )
```

EXCEPT

```
SELECT dni
```

```
FROM examen
```

Subconsultas

Las subconsultas en SQL son consultas anidadas dentro de una consulta principal. Se utilizan para realizar consultas más complejas, donde los resultados de una consulta se utilizan como entrada en otra consulta.

- = (igualdad): cuando una subconsulta retorna un único resultado, es posible compararlo contra un valor simple.
- **IN** (pertenencia): comprueba si un elemento es parte o no de un conjunto. Negación (NOT IN).
- **>ALL**: mayor que todos.
- **<=SOME**: menor o igual que alguno
- **EXISTS**: Permite comprobar si una subconsulta generó o no alguna tupla como respuesta. El resultado de la cláusula EXISTS es verdadero si la subconsulta tiene al menos una tupla, y falso en caso contrario. Negación (NOT EXIST)

Ejemplo subconsulta

Supongamos que tenemos dos tablas: "productos" y "ventas". Queremos obtener el nombre de los productos que se han vendido más de 100 unidades.

```
SELECT nombre  
FROM productos  
WHERE id IN (  
    SELECT producto_id  
    FROM ventas  
    WHERE cantidad > 100  
);
```

```
SELECT p.nombre  
FROM productos p INNER JOIN ventas v  
ON p.id = v.producto_id  
WHERE v.cantidad > 100
```


Ejemplo subconsulta

Supongamos que tenemos dos tablas: "empleados" y "salarios". Queremos obtener los empleados cuyo salario sea superior al promedio de salarios de su departamento.

```
SELECT nombre
```

```
FROM empleados e
```

```
WHERE salario > (
```

```
    SELECT AVG(salario)
```

```
    FROM salarios
```

```
    WHERE departamento_id = e.departamento_id
```

```
);
```

Ejemplo subconsulta

Supongamos que tenemos dos tablas: "clientes" y "pedidos". Queremos obtener el nombre de los clientes que hayan realizado al menos un pedido. Podemos utilizar la cláusula EXISTS en una subconsulta para lograrlo.

```
SELECT nombre
```

```
FROM clientes c
```

```
WHERE EXISTS (
```

```
    SELECT *
```

```
    FROM pedidos p
```

```
    WHERE p.cliente_id = c.id
```

```
    LIMIT 1
```

```
);
```

UPDATE ... SET, DELETE FROM

- La instrucción **DELETE FROM** se utiliza para eliminar registros de una tabla. Permite especificar una condición para filtrar los registros que se deben eliminar.

DELETE FROM clientes

WHERE edad > 50;

- La instrucción **UPDATE SET** se utiliza para actualizar los valores de uno o más registros en una tabla. Permite especificar los valores nuevos que se deben asignar a las columnas de la tabla.

UPDATE clientes

SET edad = 35

WHERE id = 1;

Ejercicios

1. Actualizar la edad del cliente de UAE a 35 años.
2. Actualizar el pais de todos los estadounidenses a EEUU.
3. Mostrar todas las órdenes de compra realizadas por clientes de 25 años.
4. Mostrar todos los clientes que no tienen órdenes de compra.
5. Mostrar el nombre completo de todos los clientes y el id de sus órdenes ordenados por order_id asi no hayan realizado ninguna compra.
6. Obtener una lista de todos los clientes junto con el número total de órdenes de compra que han realizado
7. Eliminar a los clientes que no tengan registrado ningun envio.
8. Obtener todos los clientes que hayan comprado un item con precio mayor a 300
9. Obtener los clientes que se encuentren tanto en orders como en shippings